

Name Mangling

What is Name Mangling in Python?

Name Mangling is the process in which Python internally changes the name of a private variable or method (one that starts with double underscores __). Name mangling is not true private access. It only changes the name internally.

Example 1: Private Variable

```
class Student:
```

```
    def __init__(self,name):
        self.__name = name
```

```
s = Student("John")
print(s.__Student__name)
```

Output: Poornima

What happens internally?

- We wrote self.__name but, Python internally stores it as self.__Student__name
 - So this print(s.__name) gives AttributeError: 'Student' object has no attribute '__name', because the actual name is __Student__name
-

Example 2: Private Method

```
class Student:
```

```
    def __display(self):
        print("Private Method")
```

```
s = Student()
s.__Student__display()
```

Output: Private Method

Internally Python changes __display to __Student__display

Example 3: Using dir()

```
class Employee:
```

```
    def __init__(self):
        self.__salary = 50000
```

```
print(dir(Employee))
```

Output: ['_Employee__salary',.....29 dunder methods]

Notice there is **no** `__salary`. Instead, Python stored it as `_Employee__salary`

Why is Name Mangling used?

- It is not used for security or to make variables truly private.

There are **three main reasons**:

1. Prevent Accidental Access

When an attribute starts with `__`, Python changes its name internally.

Example:

```
class Student:
    def __init__(self,name):
        self.__name = name

s = Student()
print(s.__name)
```

Output: AttributeError

Internally, Python stores it as: `_Student__name`

So users cannot accidentally access it using: `s.__name`

2. Prevent Accidental Overriding (Most Important Reason)

This is the main purpose of name mangling.

Without name mangling:

```
class Parent:
    def __init__(self, x):
        self.x = x

class Child(Parent):
    def __init__(self,x):
        super().__init__(x)
        self.x = x

c = Child(10,20)
print(c.x)
```

Output : 20

The child overrides the parent's variable.

With Name Mangling

```
class Parent:
    def __init__(self,x):
        self.__x = x

class Child(Parent):
    def __init__(self,x):
        super().__init__(x)
        self.__x = x

c = Child()
print(c._Parent__x)
print(c._Child__x)
```

Output 10

20

Because Python internally creates:

Parent -> _Parent__x

Child -> _Child__x

These are **different variables**, so the child does not accidentally overwrite the parent's private variable.

3. Avoid Naming Conflicts

Suppose many classes use the same private variable name.

```
class Car:
    def __init__(self,speed):
        self.__speed = speed

class Bike:
    def __init__(self, speed):
        self.__speed = speed
```

Internally:

Car -> _Car__speed

Bike -> _Bike__speed

Even though both classes use __speed, there is no conflict because Python stores them with different internal names.

Interview Answer

Why is Name Mangling used in Python?

- Prevent accidental access to class members.
 - Prevent accidental overriding of private members in child classes.
 - Avoid naming conflicts between parent and child classes.
-